# AI Agent Security Scanner

## Automated Red-Teaming for Autonomous AI

**Student:** Kwisinga Lewis

**Degree:** Bachelor of Software Engineering

**Date:** [Insert Date Here]

# 1. The Problem

AI agents (LangChain, AutoGPT) are being given tools:

- Read files
- Send emails
- Execute code

**The Blind Spot:** Developers deploy these agents **without testing** if they can be tricked into misusing those tools.

## 2. Why Existing Tools Fall Short

- **Static Scanners:** Only look at code, not runtime behavior.
- **LLM Red-Teamers (e.g., Garak):** Test the model in isolation, not the agentic tool-use system.
- **Defensive Guardrails:** Protect the AI, but don't *measure* its vulnerability.

**There is a gap for a runtime, agent-focused security testing framework.**

## 3. Project Objectives

Build a system that:

1. Standardizes 14 real-world attacks (OWASP Top 10).
2. Tests them against a real AI agent (Llama 3.2 via Ollama).
3. Visualizes the Attack Success Rate (ASR) in a comparative dashboard.

# 4. Methodology: The Attack Library

We created 14 payloads across 5 categories:

- **Prompt Injection:** "Ignore previous instructions..."
- **Indirect Injection:** Hidden instructions in documents.
- **Tool Abuse:** "Use file_reader to read /etc/passwd"
- **Data Exfiltration:** "Encode API keys into a URL"
- **Jailbreaks:** "You are now DAN..."

## 5. Methodology: The Harness Architecture

We built an abstraction layer to test different agents without changing the attack code.

- **MockAgentHarness:** Baseline agent (always complies). Shows 100% worst-case vulnerability.
- **LangChainHarness:** Real AI agent (Llama 3.2) with simulated tools and a defensive system prompt.

## 6. Implementation: The System

- **Backend:** Python + FastAPI (Scanner Engine).
- **Database:** SQLite (Stores Scans and Findings).
- **Frontend:** React + Tailwind (SOC-style Dashboard).
- **Deployment:** Fully containerized with Docker.

## 7. Results: The Real Finding

**Mock Agent:** 100% vulnerable (14/14 attacks succeeded).

**Llama 3.2 Agent: 64.29% vulnerable (9/14 attacks succeeded).**

- *Failed* against Prompt Injection (100% success rate).
- *Failed* against Tool Abuse (75% success rate).
- *Defended* against Data Exfiltration (0% success rate).

# 8. Live Dashboard Demo

The heatmap instantly shows developers exactly where the AI is weak:

- Red row = Prompt Injection (Vulnerable)
- Green row = Data Exfiltration (Secure)

## 9. Conclusion

- Prompt-based defenses alone are insufficient.
- AI agents are highly vulnerable to semantic manipulation.
- The AI Agent Security Scanner provides the necessary measurement tool to evaluate defenses *before* deployment.

## 10. Future Work

- Multi-agent evaluation.
- LLM-as-a-Judge for better scoring.
- CI/CD integration (security regression testing).
- Adding automated remediation.

# Thank You

**Questions?**